Communication-Avoiding Krylov Exponential Integration for Multi-GPU Option Pricing

Kevin Knights
Supervisor: Prof. Kirk M. Soodhalter

MSc. High Performance Computing
School of Mathematics, Trinity College Dublin

September 3, 2026

Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

What this talk argues

A trading desk reprices the same multi-asset PDE across a whole book, so the cost of *one* solve sets valuation throughput. That solve is dominated by **moving data**, not arithmetic, which is what *communication-avoiding* methods target.

What this talk argues

A trading desk reprices the same multi-asset PDE across a whole book, so the cost of *one* solve sets valuation throughput. That solve is dominated by **moving data**, not arithmetic, which is what *communication-avoiding* methods target.

The question is not *how* to avoid communication, but **when it is worth doing.**

What this talk argues

A trading desk reprices the same multi-asset PDE across a whole book, so the cost of *one* solve sets valuation throughput. That solve is dominated by **moving data**, not arithmetic, which is what *communication-avoiding* methods target.

The question is not *how* to avoid communication, but **when it is worth doing.**

1. **The problem and the method**
   the operator, and one exponential action per step
2. **Where the time goes**
   profiling, the two CA mechanisms, and a null result

What this talk argues

A trading desk reprices the same multi-asset PDE across a whole book, so the cost of *one* solve sets valuation throughput. That solve is dominated by **moving data**, not arithmetic, which is what *communication-avoiding* methods target.

The question is not *how* to avoid communication, but **when it is worth doing.**

1. **The problem and the method**
   the operator, and one exponential action per step
2. **Where the time goes**
   profiling, the two CA mechanisms, and a null result
3. ★ **The regime map**
   two a-priori coordinates that predict the opportunity
4. ★ **Does it pay?**
   a distributed CA integrator on four GPUs

Method in one line

Predict the opportunity *before* building the kernel, then measure whether the prediction holds.

★ marks my own contribution.

The problem: one operator, solved over and over

Three-asset Black–Scholes in log prices, backward in $\tau = T - t$, for a **basket call** and a **rainbow min-call**:

$$\frac{\partial u}{\partial \tau} = \underbrace{\tfrac{1}{2}\sum_{d,d'} \rho_{dd'}\sigma_d\sigma_{d'}\frac{\partial^2 u}{\partial x_d \partial x_{d'}}}_{\text{diffusion+correlation}} + \underbrace{\sum_{d}\left(r - \tfrac{1}{2}\sigma_d^2\right)\frac{\partial u}{\partial x_d}}_{\text{convection}} \underbrace{- ru}_{\text{reaction}} + b(\tau) \qquad (1)$$

The problem: one operator, solved over and over

Three-asset Black–Scholes in log prices, backward in $\tau = T - t$, for a **basket call** and a **rainbow min-call**:

$$\frac{\partial u}{\partial \tau} = \underbrace{\tfrac{1}{2}\sum_{d,d'} \rho_{dd'}\sigma_d\sigma_{d'}\frac{\partial^2 u}{\partial x_d \partial x_{d'}}}_{\text{diffusion+correlation}} + \underbrace{\sum_{d}\left(r - \tfrac{1}{2}\sigma_d^2\right)\frac{\partial u}{\partial x_d}}_{\text{convection}} \underbrace{- ru}_{\text{reaction}} + b(\tau) \qquad (1)$$

Second-order central differences on a uniform grid, $N = n^3$ unknowns:

- A carries **19 nonzeros per row** and $O(N_1 N_2)$ bandwidth, which rules out banded direct solvers at scale.
- Convection is skew-symmetric, so A is **non-symmetric and non-normal**.
- Stiffness grows as $\|\mathsf{A}\| \sim O(n^2)$.

The problem: one operator, solved over and over

Three-asset Black–Scholes in log prices, backward in $\tau = T - t$, for a **basket call** and a **rainbow min-call**:

$$\frac{\partial u}{\partial \tau} = \underbrace{\tfrac{1}{2}\sum_{d,d'} \rho_{dd'}\sigma_d\sigma_{d'}\frac{\partial^2 u}{\partial x_d \partial x_{d'}}}_{\text{diffusion+correlation}} + \underbrace{\sum_{d}\left(r - \tfrac{1}{2}\sigma_d^2\right)\frac{\partial u}{\partial x_d}}_{\text{convection}} \underbrace{- ru}_{\text{reaction}} + b(\tau) \qquad (1)$$

Second-order central differences on a uniform grid, $N = n^3$ unknowns:

- A carries **19 nonzeros per row** and $O(N_1 N_2)$ bandwidth, which rules out banded direct solvers at scale.
- Convection is skew-symmetric, so A is **non-symmetric and non-normal**.
- Stiffness grows as $\|\mathrm{A}\| \sim O(n^2)$.

Production resolution is $n = 61$: **226,981 unknowns**, 4.23 M nonzeros.
Every timing in this talk is for the **basket call**, the more expensive of the two.

The method: one exponential action per time step

Discretizing space only leaves time continuous: the PDE becomes a large linear system of ODEs, one equation per grid point.

$$\frac{d\mathsf{u}}{d\tau} = \mathsf{A}\mathsf{u} + \mathsf{B}\mathsf{s}(\tau), \qquad \mathsf{u}(0) = \mathsf{u}_0 \quad \text{(the payoff)}. \tag{2}$$

The method: one exponential action per time step

Discretizing space only leaves time continuous: the PDE becomes a large linear system of ODEs, one equation per grid point.

$$\frac{d\mathsf{u}}{d\tau} = \mathsf{A}\mathsf{u} + \mathsf{B}\mathsf{s}(\tau), \qquad \mathsf{u}(0) = \mathsf{u}_0 \quad \text{(the payoff)}. \tag{2}$$

For the basket, the payoff and the far-field boundary forcing are

$$\mathsf{u}_0(\mathsf{x}) = \max\Big(\textstyle\sum_d w_d e^{x_d} - K,\ 0 \Big), \qquad \mathsf{s}(\tau) = \big[\tfrac{\tau^2}{2},\ \tau,\ 1 \big]^{\mathsf{T}}.$$

Discretizing space only leaves time continuous: the PDE becomes a large linear system of ODEs, one equation per grid point.

$$\frac{d\mathsf{u}}{d\tau} = \mathsf{A}\mathsf{u} + \mathsf{B}\mathsf{s}(\tau), \qquad \mathsf{u}(0) = \mathsf{u}_0 \quad \text{(the payoff)}. \tag{2}$$

For the basket, the payoff and the far-field boundary forcing are

$$\mathsf{u}_0(\mathsf{x}) = \max\Big(\textstyle\sum_d w_d e^{x_d} - K,\ 0 \Big), \qquad \mathsf{s}(\tau) = \big[\tfrac{\tau^2}{2},\ \tau,\ 1 \big]^{\mathsf{T}}.$$

The forcing is polynomial, so $\mathsf{s}' = \mathsf{K}\mathsf{s}$ with K nilpotent. Stacking the two states absorbs it *exactly*, and (2) loses its forcing term altogether:

$$\widetilde{\mathsf{A}} = \begin{bmatrix} \mathsf{A} & \mathsf{B} \\ 0 & \mathsf{K} \end{bmatrix} \in \mathbb{R}^{(N+3)\times(N+3)}, \quad \widetilde{\mathsf{v}} = \begin{bmatrix} \mathsf{u} \\ \mathsf{s} \end{bmatrix} \implies \frac{d\widetilde{\mathsf{v}}}{d\tau} = \widetilde{\mathsf{A}}\widetilde{\mathsf{v}}. \tag{3}$$

Discretizing space only leaves time continuous: the PDE becomes a large linear system of ODEs, one equation per grid point.

$$\frac{d\mathsf{u}}{d\tau} = \mathsf{A}\mathsf{u} + \mathsf{B}\mathsf{s}(\tau), \qquad \mathsf{u}(0) = \mathsf{u}_0 \quad \text{(the payoff)}. \tag{2}$$

For the basket, the payoff and the far-field boundary forcing are

$$\mathsf{u}_0(\mathsf{x}) = \max\Big(\textstyle\sum_d w_d e^{x_d} - K,\ 0 \Big), \qquad \mathsf{s}(\tau) = \big[\tfrac{\tau^2}{2},\ \tau,\ 1 \big]^{\mathsf{T}}.$$

The forcing is polynomial, so $\mathsf{s}' = \mathsf{K}\mathsf{s}$ with K nilpotent. Stacking the two states absorbs it *exactly*, and (2) loses its forcing term altogether:

$$\widetilde{\mathsf{A}} = \begin{bmatrix} \mathsf{A} & \mathsf{B} \\ 0 & \mathsf{K} \end{bmatrix} \in \mathbb{R}^{(N+3)\times(N+3)}, \quad \widetilde{\mathsf{v}} = \begin{bmatrix} \mathsf{u} \\ \mathsf{s} \end{bmatrix} \implies \frac{d\widetilde{\mathsf{v}}}{d\tau} = \widetilde{\mathsf{A}}\widetilde{\mathsf{v}}. \tag{3}$$

One step from τ_k is then *one matrix-exponential action* on $\widetilde{\mathsf{v}}_k$, approximated in the Krylov subspace $\mathcal{K}_m(\mathsf{G}, \widetilde{\mathsf{v}}_k)$, $\mathsf{G} = h\widetilde{\mathsf{A}}$:

$$\mathsf{u}_{k+1} = e^{\mathsf{G}}\widetilde{\mathsf{v}}_k \ \approx\ \beta\, \mathsf{V}_m\, e^{\mathsf{H}_m} \mathsf{e}_1, \qquad \beta = \|\widetilde{\mathsf{v}}_k\|. \tag{4}$$

Call this solver **KSM-EI**: a **K**rylov **s**ubspace **m**ethod used as an **e**xponential **i**ntegrator.

1. The problem and the method
2. Where the time goes
3. The regime map
4. Does it pay?

Profiling: two kernels, two different ceilings

Kernel	n=31	n=61
SpMV	83.5%	78.0%
MGS	11.4%	16.6%
dense $\exp(H_m)$	0.40%	0.23%
mean m	7.60	8.91

share of KSM-EI solve time

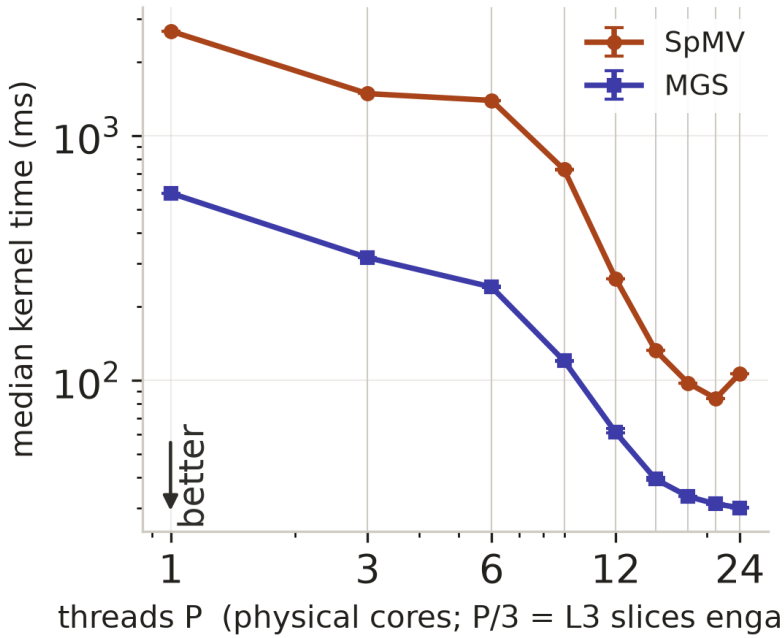
Grids follow Niesen & Wright's option-pricing study, so the semi-discrete ODE error stays comparable with theirs.

Profiling: two kernels, two different ceilings

Kernel	n=31	n=61
SpMV	83.5%	78.0%
MGS	11.4%	16.6%
dense exp(H_m)	0.40%	0.23%
mean m	7.60	8.91

share of KSM-EI solve time

Grids follow Niesen & Wright's option-pricing study, so the semi-discrete ODE error stays comparable with theirs.



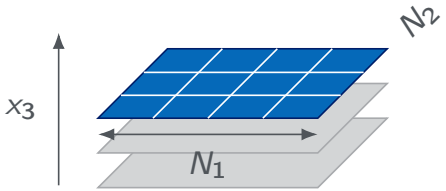
Strong scaling at n=61: fixed work, more cores.

SpMV plateaus until the working set fits the engaged cache. Modified Gram–Schmidt (MGS) keeps improving, but every inner product it performs is a global reduction, and its share grows with m.

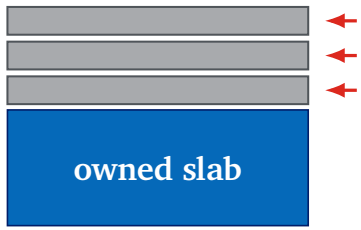
The obstacle. Arnoldi iteration j needs v_j before it can start: one SpMV and one global reduction per step, strictly in sequence.

The obstacle. Arnoldi iteration j needs v_j before it can start: one SpMV and one global reduction per step, strictly in sequence.

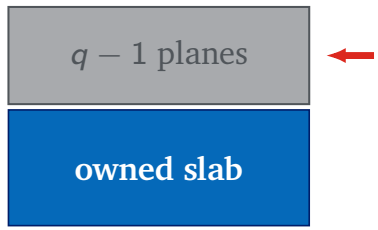
1. Vertical: one deep exchange, not $q - 1$.



one **plane**:
an $N_1 \times N_2$ cut



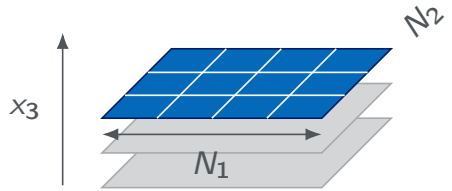
naive:
$q - 1$ messages



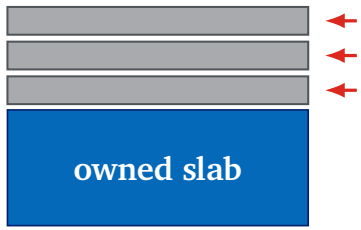
deep halo:
1 message, same bytes

The obstacle. Arnoldi iteration j needs v_j before it can start: one SpMV and one global reduction per step, strictly in sequence.

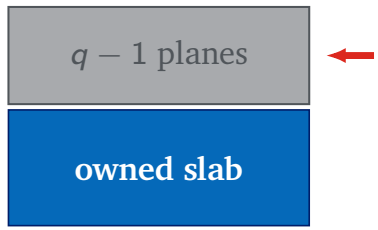
1. Vertical: one deep exchange, not $q-1$.



one **plane**:
an $N_1 \times N_2$ cut



naive:
$q-1$ messages



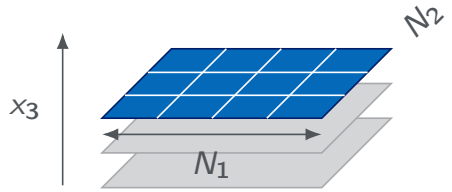
deep halo:
1 message, same bytes

2. Horizontal: one block, not m vectors. Orthogonalize a whole Krylov block at once with **CholeskyQR2**: form the Gram matrix $Z^T Z$, Cholesky-factor it, repeat once for stability.

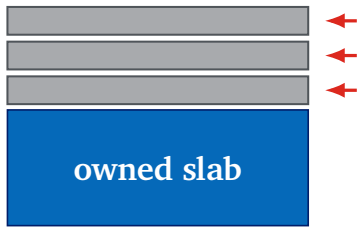
$$Z_q = [v,\ Av,\ \ldots,\ A^{q-1}v] \qquad \underbrace{1 + \frac{m(m+3)}{2}}_{\text{MGS}} \longrightarrow \underbrace{1 + 2\lceil m/q \rceil}_{\text{one block}}$$

The obstacle. Arnoldi iteration j needs v_j before it can start: one SpMV and one global reduction per step, strictly in sequence.

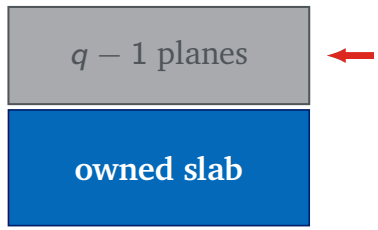
1. Vertical: one deep exchange, not $q-1$.



one **plane**:
an $N_1 \times N_2$ cut



naive:
$q-1$ messages



deep halo:
1 message, same bytes

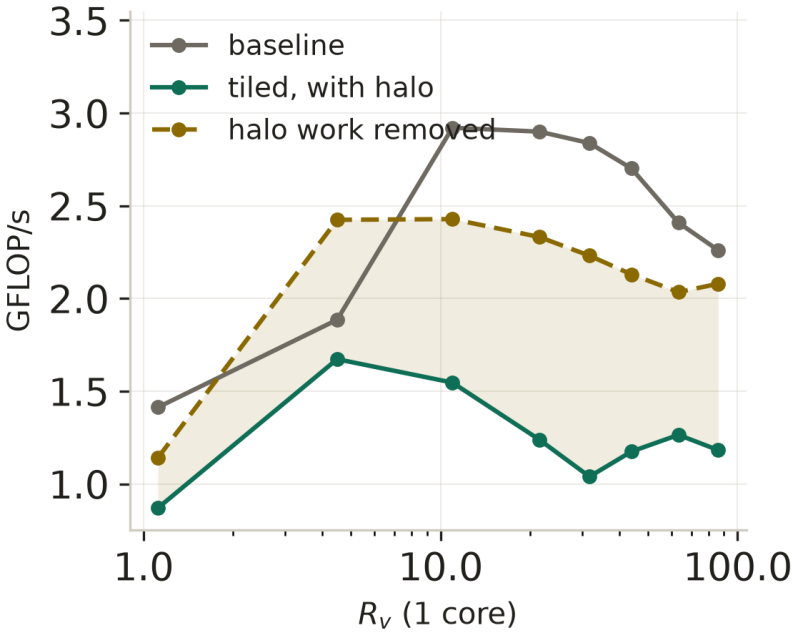
2. Horizontal: one block, not m vectors. Orthogonalize a whole Krylov block at once with **CholeskyQR2**: form the Gram matrix $Z^T Z$, Cholesky-factor it, repeat once for stability.

$$Z_q = [v,\ Av,\ \ldots,\ A^{q-1}v] \qquad \underbrace{1 + \frac{m(m+3)}{2}}_{\text{MGS}} \longrightarrow \underbrace{1 + 2\lceil m/q \rceil}_{\text{one block}}$$

Messages fall from $q-1$ to one and reductions by **11–16**$\times$ at the measured $m = 8$–11, $q = 8$; the price is recomputing the ghost region.

On puffin it does not pay

Both mechanisms built and timed on *puffin*, a 24-core Threadripper 3960X on a single NUMA socket.

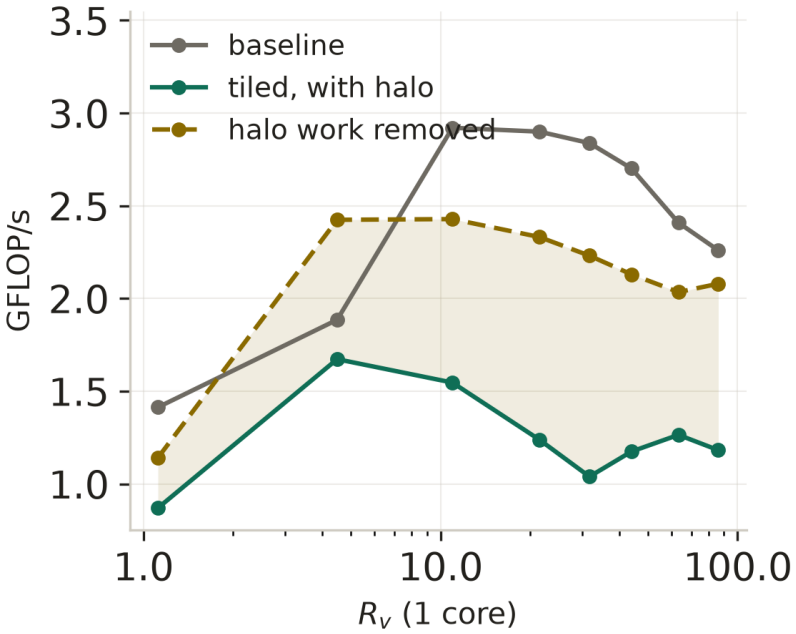


Vertical (L2): the gap is the halo.

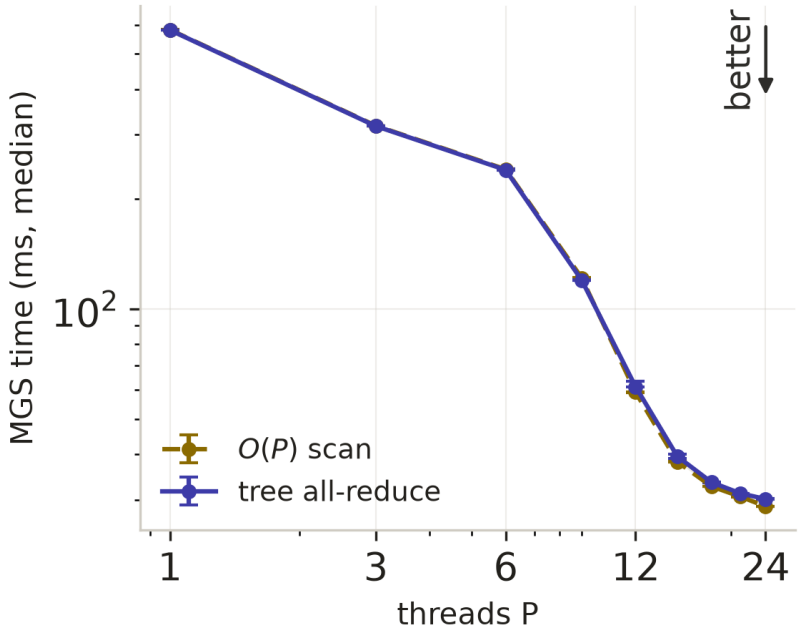
The L3 sweep agrees.

On puffin it does not pay

Both mechanisms built and timed on *puffin*, a 24-core Threadripper 3960X on a single NUMA socket.



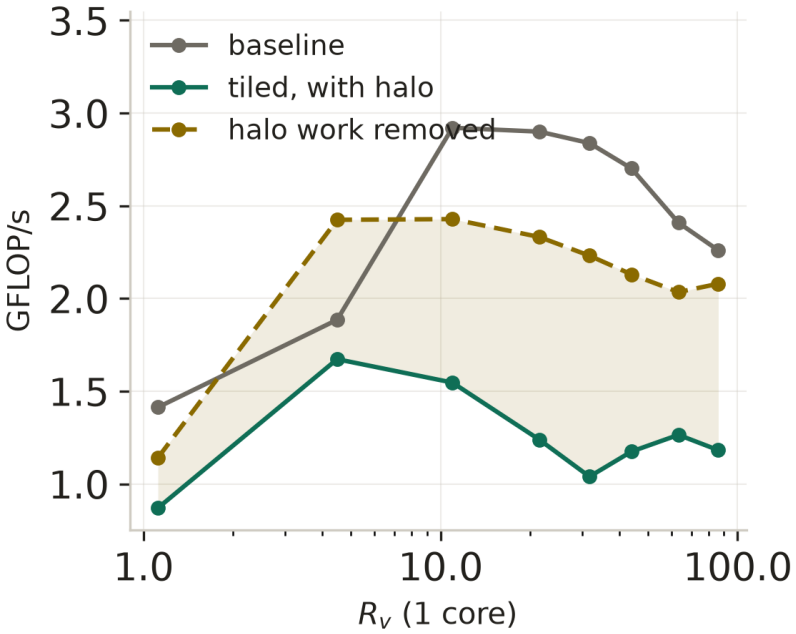
Vertical (L2): the gap is the halo. The L3 sweep agrees.



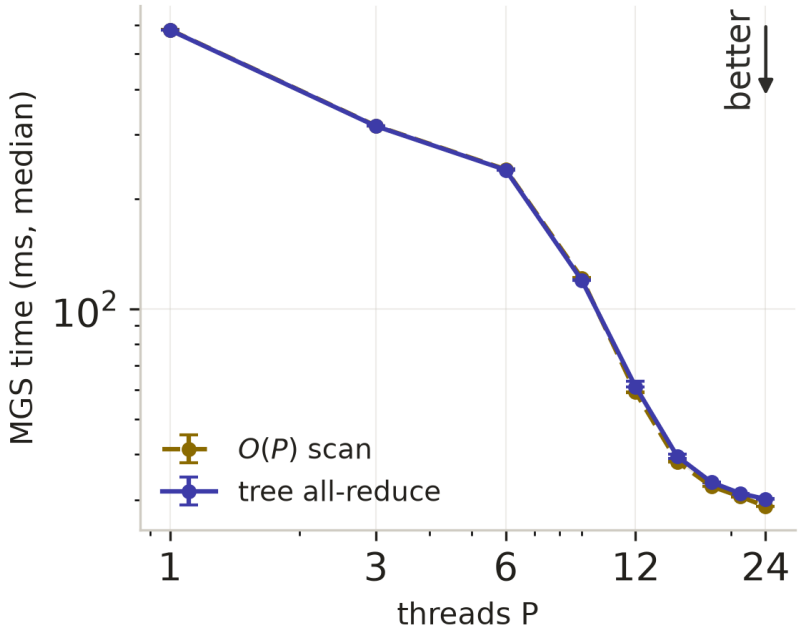
Horizontal: a 1.7× faster all-reduce leaves MGS within 5%.

On puffin it does not pay

Both mechanisms built and timed on *puffin*, a 24-core Threadripper 3960X on a single NUMA socket.



Vertical (L2): the gap is the halo. The L3 sweep agrees.



Horizontal: a 1.7× faster all-reduce leaves MGS within 5%.

11–16× fewer reductions, 4–8× less modeled traffic.
The saving is real; the speedup is not. So when *does* it pay?

1. The problem and the method
2. Where the time goes
3. The regime map
4. Does it pay?

The regime map: predict first, then measure

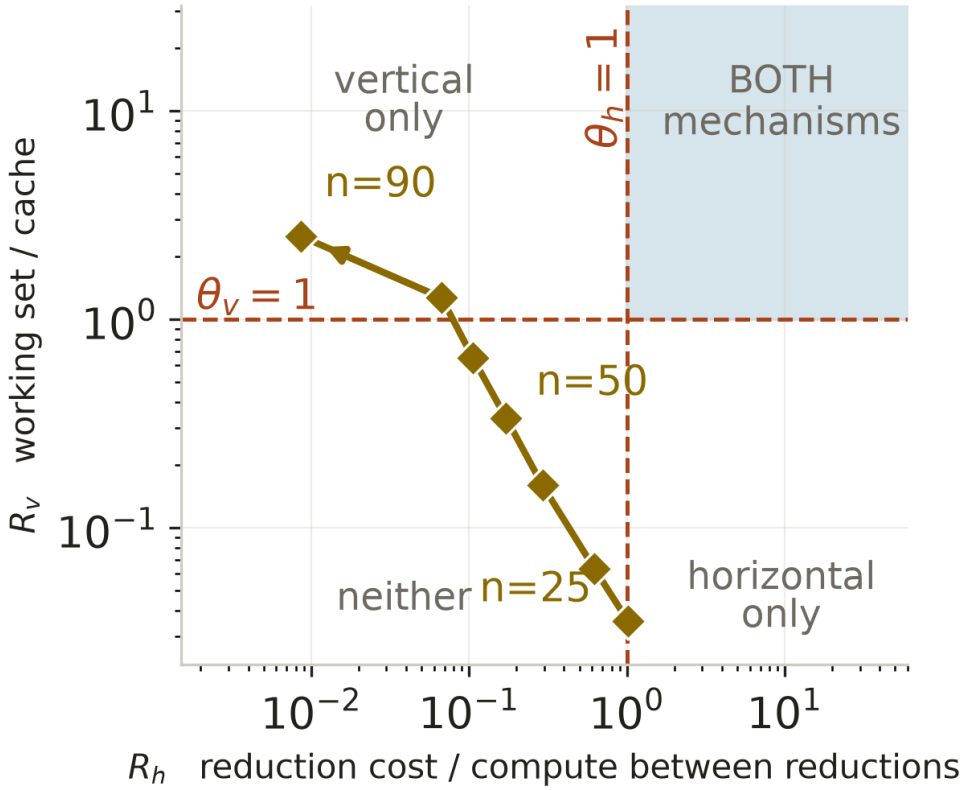
$$R_v = \frac{\text{Arnoldi working set}}{\text{last-level cache}}$$

The regime map: predict first, then measure

$$R_v = \frac{\text{Arnoldi working set}}{\text{last-level cache}} \qquad R_h = \frac{\text{reduction latency}}{\text{compute between reductions}}$$

The regime map: predict first, then measure

$$R_v = \frac{\text{Arnoldi working set}}{\text{last-level cache}} \qquad R_h = \frac{\text{reduction latency}}{\text{compute between reductions}}$$



Eight production grids, $n = 25 \to 90$, real basket operator, puffin's calibrated constants.

On one socket $R_v R_h$ stays near 0.04 for every P, so the corner is not merely unreached here: it is unreachable on this topology.

R_v, **from the problem.** Working-set bytes implied by N, the sparsity pattern and the measured Krylov dimension m, over the last-level cache actually available to the team:

$$R_v = \frac{\text{bytes}(N,\ \text{nnz},\ m)}{C_{\text{LLC}}(P)}.$$

R_v, **from the problem.** Working-set bytes implied by N, the sparsity pattern and the measured Krylov dimension m, over the last-level cache actually available to the team:

$$R_v = \frac{\text{bytes}(N,\ \text{nnz},\ m)}{C_{\text{LLC}}(P)}.$$

R_h, **from a calibration instrument.** An isolated scalar all-reduce with no application work, fitted over the tree levels it crosses:

$$R_h = \frac{t_{\text{red}}(P)}{W_{\text{local}}}, \qquad t_{\text{red}}(P) = t_{\text{intra}}\, L_{\text{in}}(P) + t_{\text{cross}}\, \min\big(L_{\text{x}}(P),\ L^{\star}\big).$$

W_{local}, compute between reductions; t_{intra}, t_{cross}, cost of one tree level inside and across a cache domain; L_{in}, L_{x}, how many levels of each kind the tree crosses; $L^{\star}$, fitted depth beyond which concurrency hides the cost.

R_v, **from the problem.** Working-set bytes implied by N, the sparsity pattern and the measured Krylov dimension m, over the last-level cache actually available to the team:

$$R_v = \frac{\text{bytes}(N,\ \text{nnz},\ m)}{C_{\text{LLC}}(P)}.$$

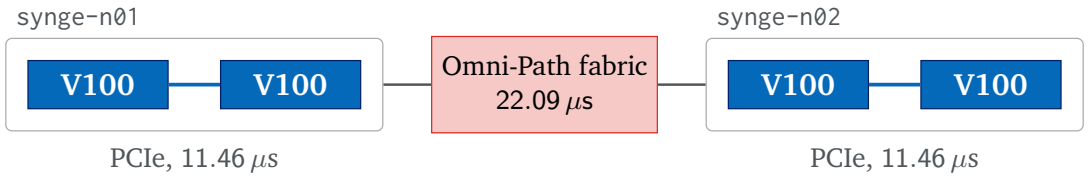
R_h, **from a calibration instrument.** An isolated scalar all-reduce with no application work, fitted over the tree levels it crosses:

$$R_h = \frac{t_{\text{red}}(P)}{W_{\text{local}}}, \qquad t_{\text{red}}(P) = t_{\text{intra}}\, L_{\text{in}}(P) + t_{\text{cross}}\, \min\big(L_{\text{x}}(P),\ L^{\star}\big).$$

W_{local}, compute between reductions; t_{intra}, t_{cross}, cost of one tree level inside and across a cache domain; L_{in}, L_{x}, how many levels of each kind the tree crosses; $L^{\star}$, fitted depth beyond which concurrency hides the cost.

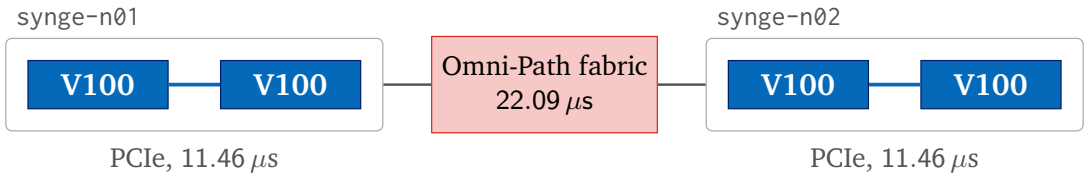
Raising the cost of a reduction is what moves an operator right, and that is a property of topology.

More participants push the operator into the corner

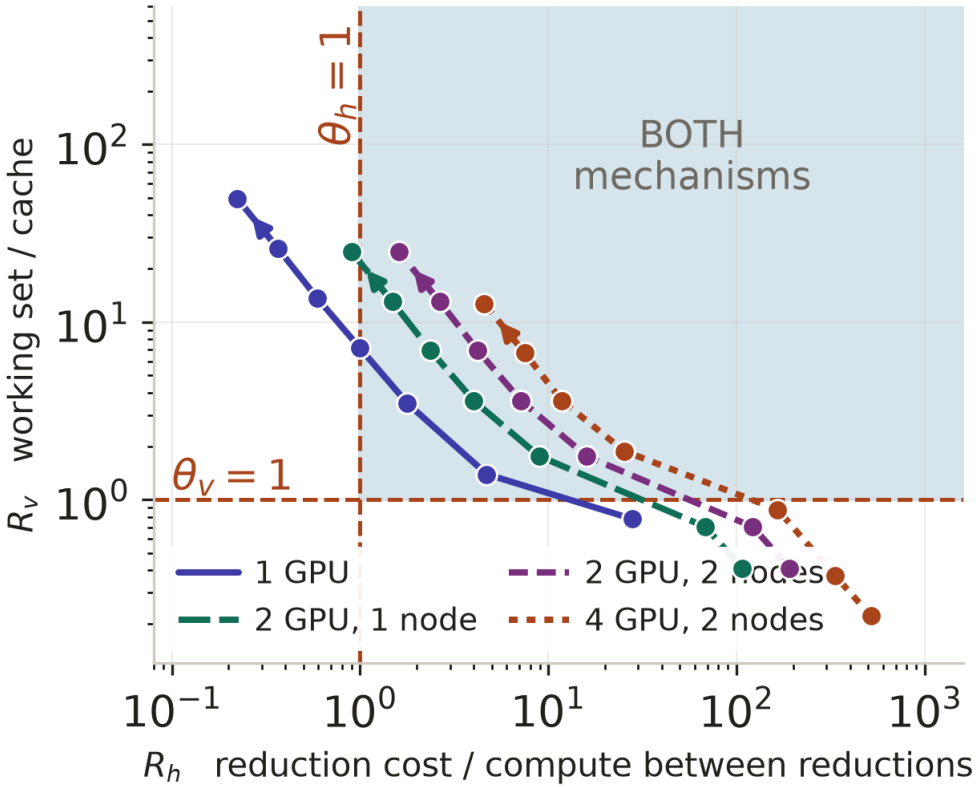


A reduction inside a card costs tens of nanoseconds; one that leaves the node costs **22 μs**.

More participants push the operator into the corner



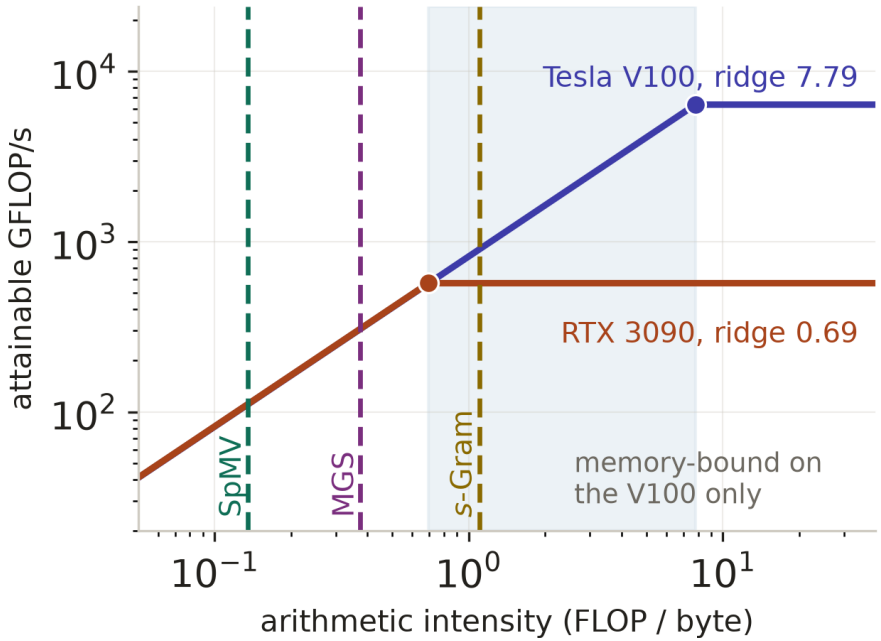
A reduction inside a card costs tens of nanoseconds; one that leaves the node costs **22 μs**.



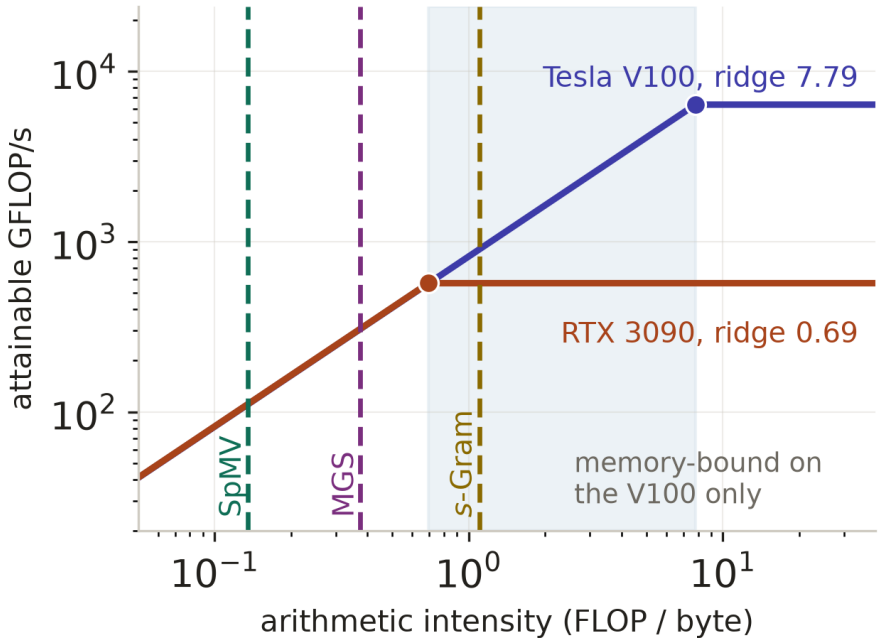
Every added participant raises t_{red}, moving the whole trajectory **right**.

At n=61, m=12, q=8: $R_v = 7.2$, $R_h = 1.7$, inside the corner on both cards.

Reaching the corner is not enough: the device's roofline



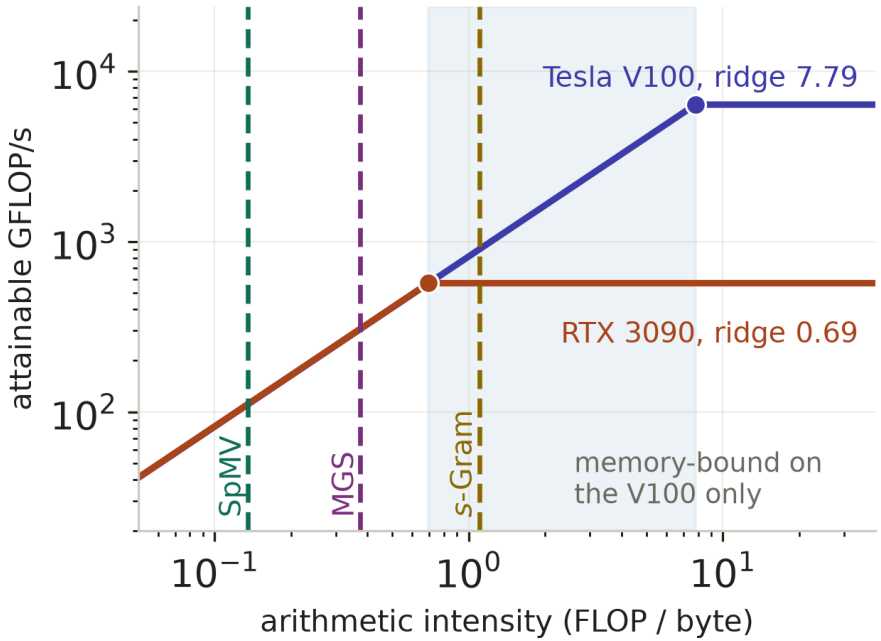
Reaching the corner is not enough: the device's roofline



Nsight Compute, *s*-Gram kernel

% of roof	V100	3090
DRAM	**86%**	38%
compute	16%	**94%**
verdict	memory	compute

Reaching the corner is not enough: the device's roofline



Nsight Compute, *s*-Gram kernel

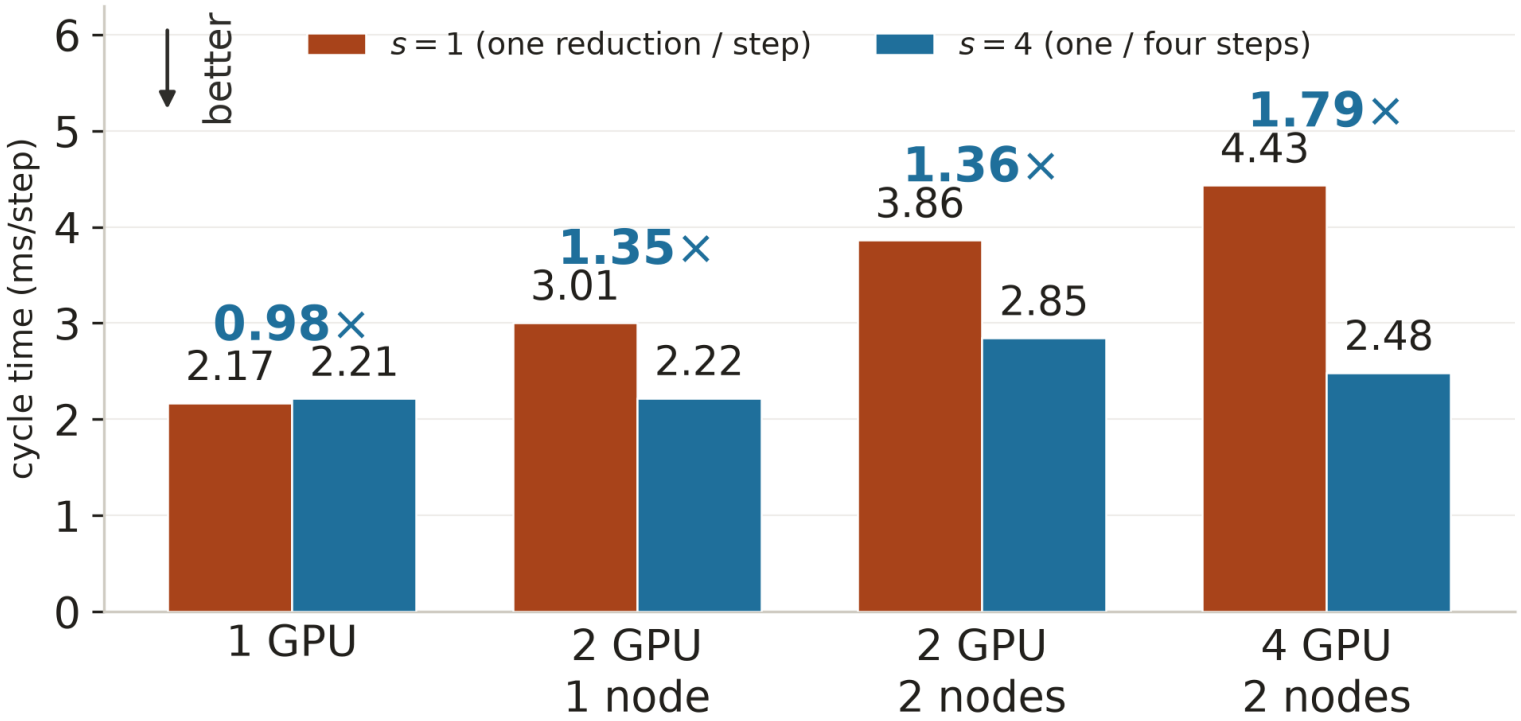
% of roof	V100	3090
DRAM	**86%**	38%
compute	16%	**94%**
verdict	memory	compute

The memory-bound check

A precondition, before the map is read: compare each kernel's arithmetic intensity (AI, FLOP per byte) with the FP64 ridge, peak FP64 over achieved bandwidth (BW).

$$AI_{\text{kernel}} < \underbrace{\frac{\text{peak FP64}}{BW_{\text{achieved}}}}_{\text{FP64 ridge}} \implies \text{memory-bound, and on the map.}$$

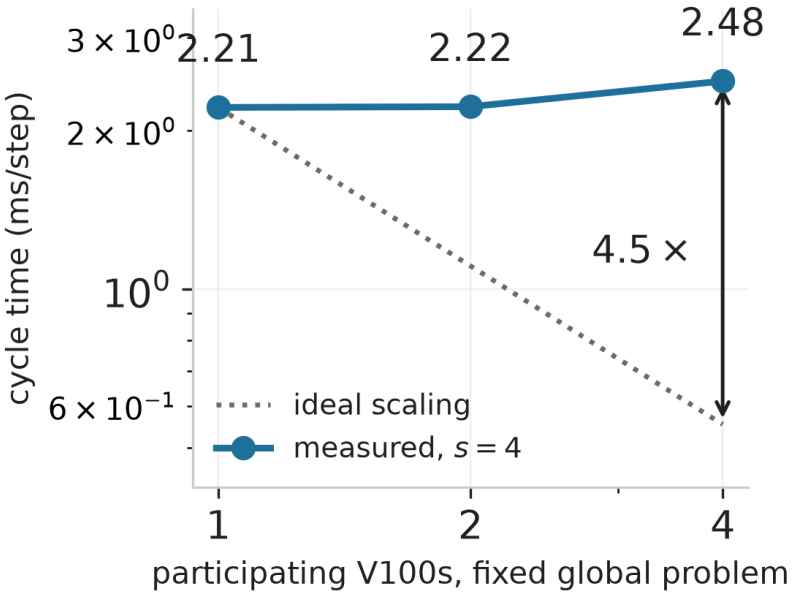
Does communication avoidance pay?



Basket, production grid n=61, 100 steps, *exact-depth* halo: both widths do equal numerical work.

Parity on one GPU, and **1.78–1.80**× on four V100s across two nodes.

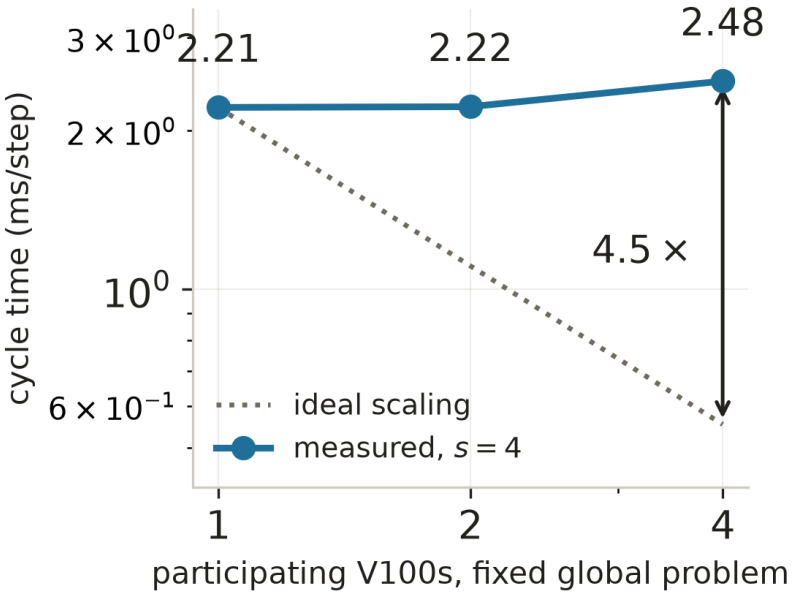
The honest limit: CA helps, distribution still costs



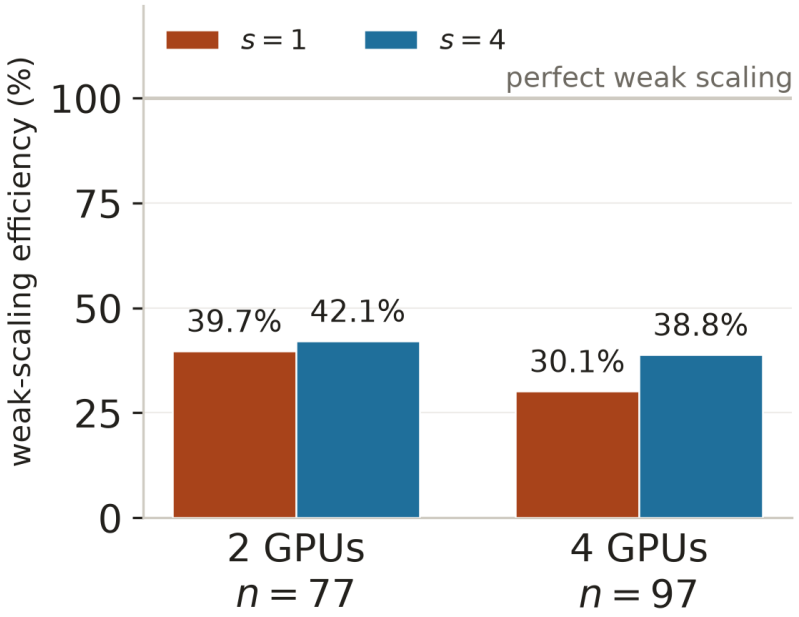
Strong: fixed problem, more GPUs.

0.89×, about 22% efficiency.

The honest limit: CA helps, distribution still costs

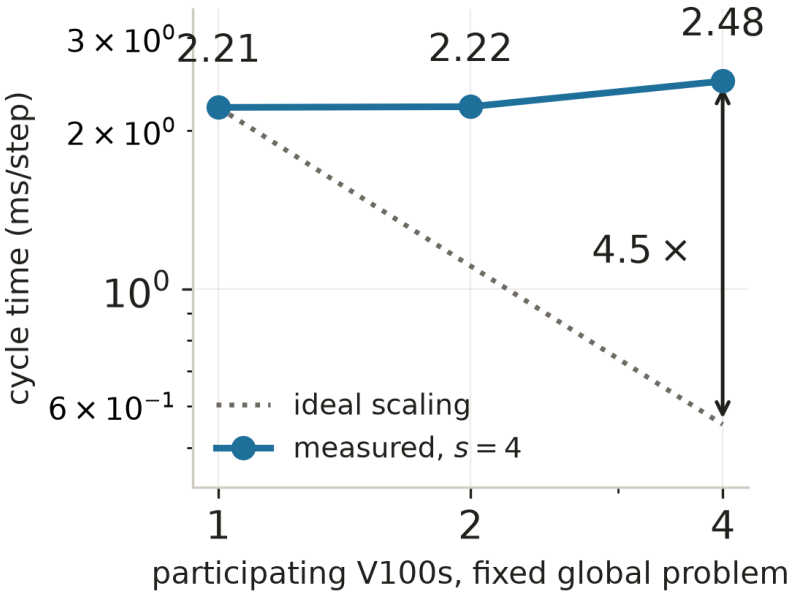


Strong: fixed problem, more GPUs.
0.89×, about 22% efficiency.

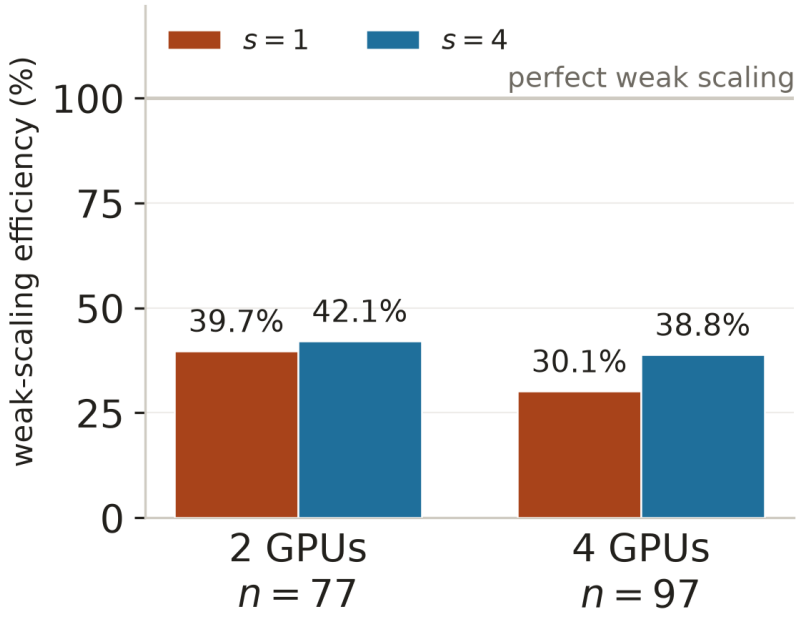


Weak: ≈227,000 rows per GPU held fixed.

The honest limit: CA helps, distribution still costs



Strong: fixed problem, more GPUs.
0.89×, about 22% efficiency.



Weak: ≈227,000 rows per GPU held fixed.

At n=61 each of four slabs owns only ≈57,000 rows: too little to saturate a V100. Four GPUs earn their place **above** $n = 337$, where the problem stops fitting on one card.

Conclusions and future work

Communication avoidance pays only when the **operator**, the **kernel** and the **machine** all line up.

Communication avoidance pays only when the **operator**, the **kernel** and the **machine** all line up.

What the work establishes

- Two a-priori coordinates that say *which* mechanism is worth measuring, without timing the solver.
- One system where the corner is unreachable and one where more participants reach it, on the same definitions.
- A distributed CA integrator, **1.78–1.80**× faster on four V100s, validated against independent references and **5.8–6.4**× faster than smoothed ADI at equal declared accuracy.

Communication avoidance pays only when the **operator**, the **kernel** and the **machine** all line up.

What the work establishes

- Two a-priori coordinates that say *which* mechanism is worth measuring, without timing the solver.
- One system where the corner is unreachable and one where more participants reach it, on the same definitions.
- A distributed CA integrator, **1.78–1.80**× faster on four V100s, validated against independent references and **5.8–6.4**× faster than smoothed ADI at equal declared accuracy.

Future work

- A **tile-aware** R_v predicting read redundancy from tile dimensions, stencil radius and streamed height.
- **Mixed precision** recurrence with higher-precision Gram accumulation. Not free: it can cost numerical stability, so the first job is identifying which parts of the solve are sensitive to the change.
- **H100-class devices and faster links**: does the crossover transfer beyond Synge?

This work stands on a great deal of help.

My thanks to my supervisor, to the staff and students of the MSc in High-Performance Computing, to everyone who kept the clusters running, and to my family, for the teaching, the patience and the support that made it possible.

"If I have seen further it is by standing on the shoulders of Giants."

Isaac Newton, letter to Robert Hooke, February 1675

Questions?

Code: `github.com/kevinknights29/caksm`

Thesis: `.../caksm/docs/thesis/main.pdf`

Appendix

Appendix: financial numerical validation

Are the prices still right, and how does the solver compare with an established method?
KSM-EI is the Krylov exponential integrator.
ADI-HV-S the smoothed Hundsdorfer–Verwer ADI scheme.

Equal declared accuracy, one thread, n=61

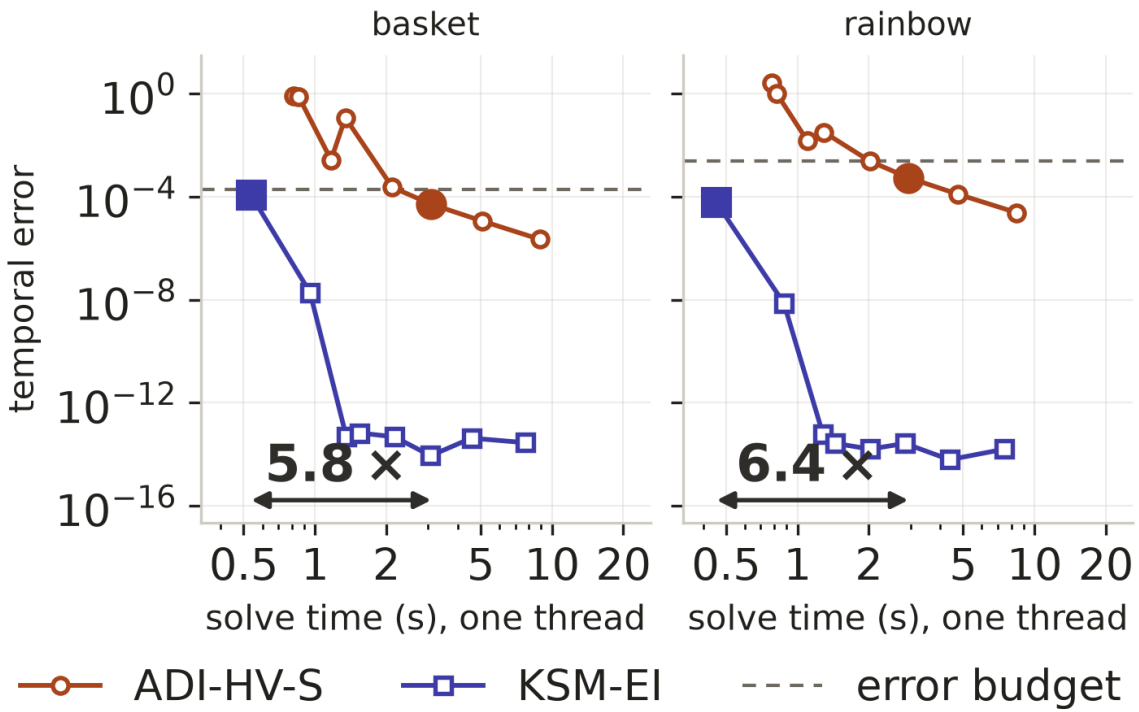
Payoff	Method	Steps	Median (s)	
Basket	ADI-HV-S	50	3.090	
Basket	KSM-EI	1	0.533	**5.8**×
Rainbow	ADI-HV-S	50	2.936	
Rainbow	KSM-EI	1	0.460	**6.4**×

Where the price error actually is

Source of Δ	Basket	Rainbow
reference	6.8×10^{-6}	$<10^{-13}$
domain	1.0×10^{-4}	4.5×10^{-6}
spatial	2.9×10^{-3}	3.6×10^{-2}
ADI temporal	2.2×10^{-6}	2.4×10^{-5}
KSM temporal	5.3×10^{-15}	3.6×10^{-15}

Both solvers refine at second order against independent references.
Basket Δ errors are 0.011–0.020%. The ADI comparison is **Dang, Christara & Jackson (2010)**, my last seminar paper.

Appendix: where the 5.8× comes from



One thread, n=61; each marker is one step count, 1 step at the left to 400 at the right.

ADI-HV-S needs **50 steps** to meet the error budget. KSM-EI reaches machine precision in **one**.

ADI-HV-S is second order in time, so accuracy has to be bought with steps. KSM-EI's curve is vertical: past the first step there is no temporal error left to remove, and further steps buy only time.

Appendix: is the Krylov block numerically admissible?

A monomial block $[v, Av, \ldots, A^{q-1}v]$ loses conditioning fast, so CholeskyQR2 needs a guard.

In practice. The solver narrows the block whenever $\kappa_2(Z_q) > u^{-1/2} \approx 9.5 \times 10^7$.

In theory. CholeskyQR2's sufficient condition is dimension-dependent,

$$\delta = 8\kappa_2(Z_q)\sqrt{(N+3)qu + q(q+1)u} \le 1,$$

which allows only $\kappa_2 \lesssim 1.25 \times 10^4$ at our size. The measured block reaches 6.0×10^6, so $\delta \approx 482$: the theorem is **inconclusive here, not violated**.

So the claim rests on the guard, a successful factorization, the orthogonality measured on the right, and referee agreement.

Orthogonality loss, n=61, m=8, q=4

Method	$\|I - Q^T Q\|$	Time
CholeskyQR2	4.0×10^{-14}	1.00×
TSQR	3.5×10^{-14}	3.06×
Block GS, reorth.	6.5×10^{-14}	1.24×

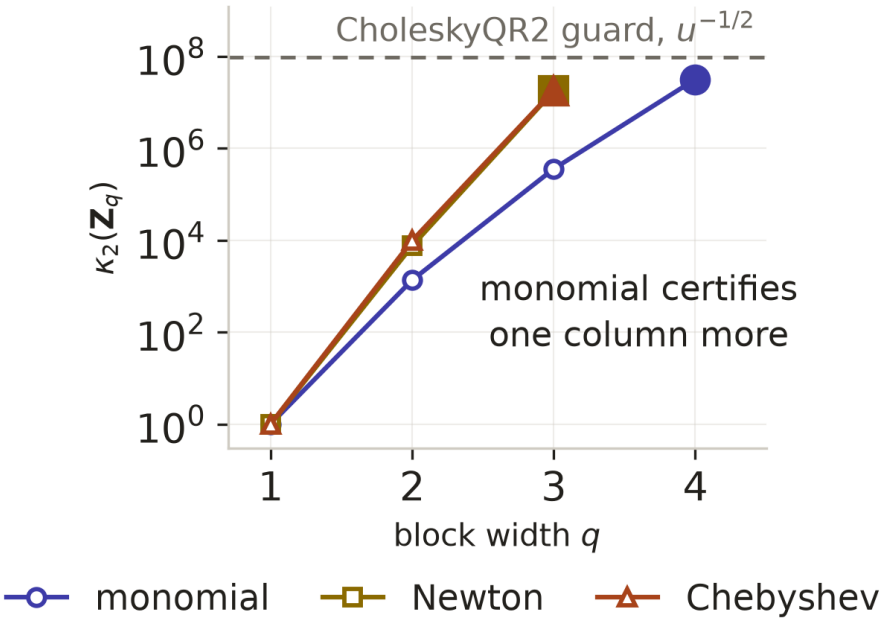
lower is better in both columns

Width admitted by the guard

Basis	n=31	n=61
Monomial	**4**	**4**
Newton (Leja)	3	3
Chebyshev	4	3

higher is better: a wider block means fewer reductions

Appendix: why the monomial basis is enough



Basket, n=61. Each line stops at the widest block that basis certifies (filled marker).

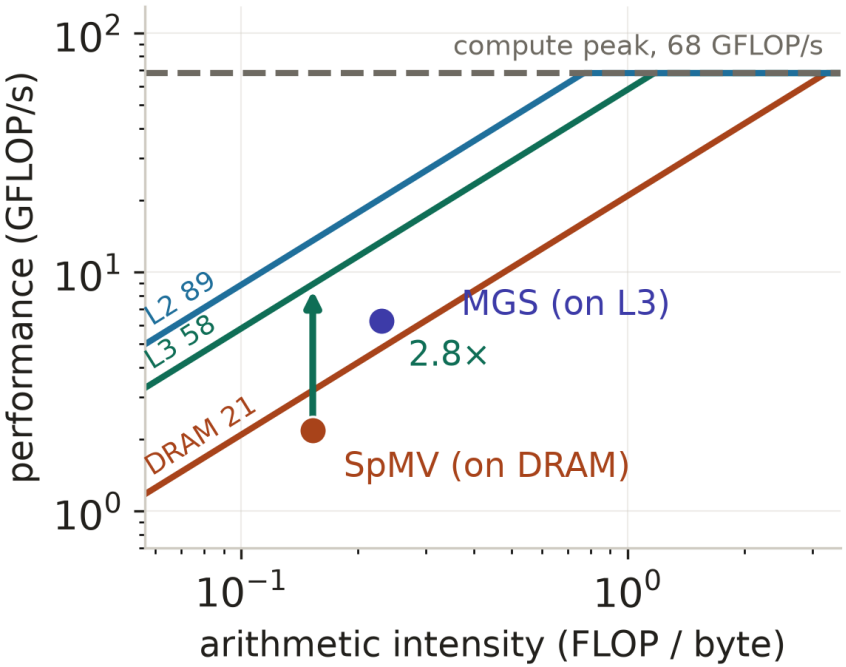
m stays small. The endpoint defect is met at a mean $m \approx 8$, so the recurrence never needs $q > 4$.

Small q is the safe regime. Monomial conditioning is only a problem for wide blocks. At $q \leq 4$ it stays under the guard, and grows *more slowly* than either alternative.

The alternatives do not pay. Newton and Chebyshev exist to hold conditioning down at large q. Here they are worse at every width, certify one column fewer, and need spectral estimates the monomial does not.

Stability here comes from m being small, not from a clever basis.

Appendix: the cache-aware roofline on puffin



Kernel	*AI*	**Tier**
SpMV	0.15	DRAM
MGS	0.23	L3
dense exp(H_m)	1.76	L2

arithmetic intensity, FLOP per byte

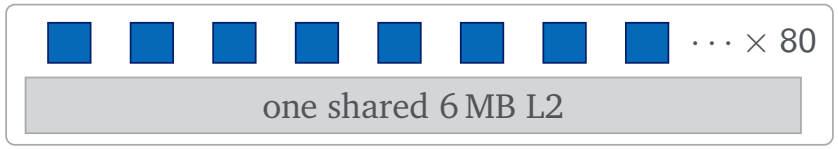
One core, n=61.

$$\text{gain} = \frac{\min(BW_{\text{fast}} \cdot AI,\ \text{peak})}{\min(BW_{\text{slow}} \cdot AI,\ \text{peak})} = \frac{58.2}{20.9} = 2.8\times \quad \text{at } AI = 0.15.$$

Appendix: where a GPU differs from a CPU

CPU: puffin, Threadripper 3960X

GPU: synge, Tesla V100



CPU

- Cache **grows with the team**: each engaged core complex brings its own L3 slice.
- R_v scales with the engaged core count P, so the two axes are coupled and $R_v R_h$ is roughly P-independent.
- Reductions cross **2** levels.

GPU

- Cache is **fixed**: engaging more SMs adds none.
- R_v stops depending on how many SMs are engaged, so **grid size alone moves it**.
- Reductions cross **5** levels, one of them a kernel launch. Coalescing also makes the access pattern far more sensitive.

Appendix: the reduction ladder and $t^{\star}_{\mathrm{red}}$

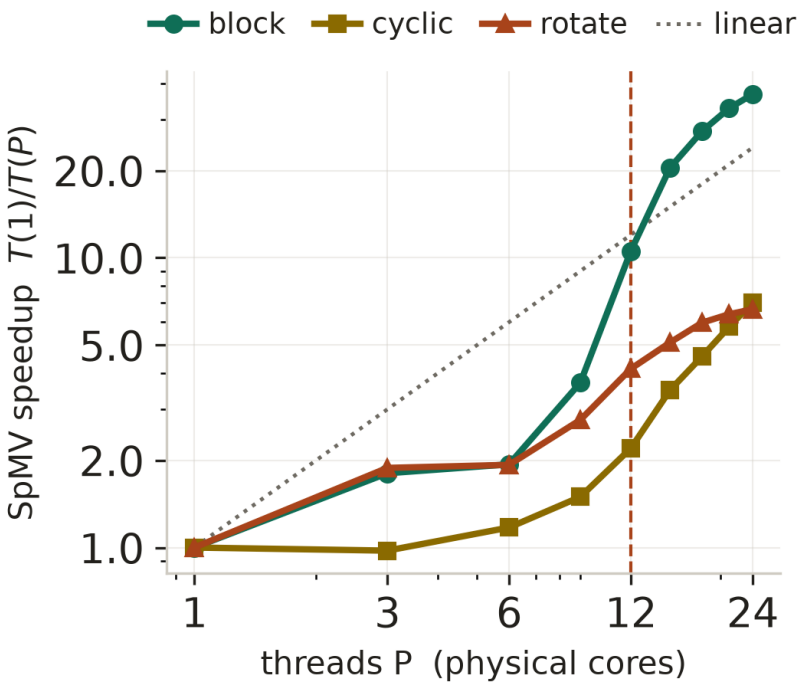
$$R_v R_h = g(m)\, R(m)\, \Lambda, \qquad \Lambda = \frac{t_{\mathrm{red}} \cdot BW}{C} \qquad \Longrightarrow \qquad t^{\star}_{\mathrm{red}} = 0.69\,\mu s$$

priced at the production point n=61, m=17.

Reduction level	t_{red} **(μs)**	**Corner**
warp shuffle	0.42	closed
thread block	0.79	**open**
grid, 2nd launch	5.54	**open**
device, PCIe	11.46	**open**
node, fabric	22.09	**open**

Puffin, for comparison: $t_{\mathrm{intra}} = 120.8\,\mathrm{ns}$, $t_{\mathrm{cross}} = 652.8\,\mathrm{ns}$, $L^{\star} = 2.20$, $R^2 = 0.98$.

Appendix: on the CPU, memory access is the binding constraint



Same operator and byte count;
only the row-to-core map changes.

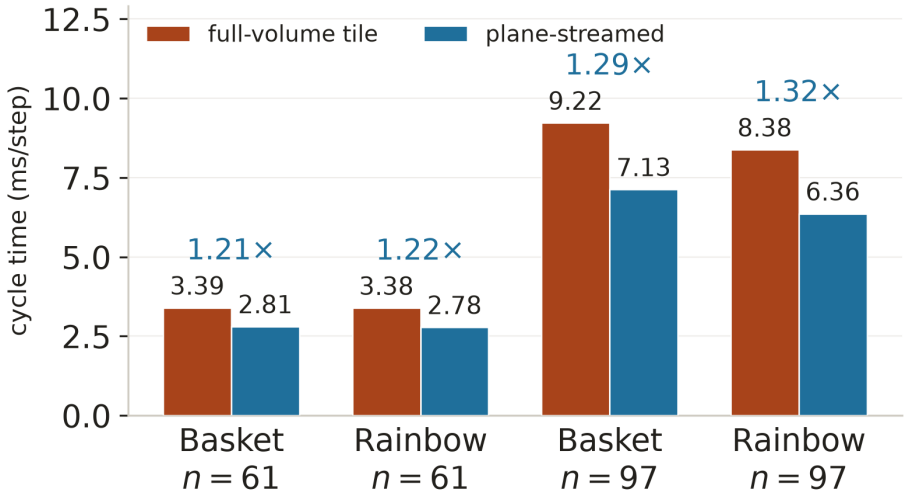
schedule	Rows per core	Speedup
`block`	contiguous band	**36.7**×
`cyclic`	same bytes, dealt out	7.0×
`rotate`	contiguous, reshuffled	6.6×

the OpenMP `schedule` clause

At the knee the implied 176 GB/s exceeds puffin's ∼95 GB/s DRAM roof, so it comes from L3. **Contiguity, not volume, keeps a slice resident.**

On one NUMA node no reduction costs more than a core-complex hop, so synchronization is never dominant: the lever here is **access order**, not communication avoidance.

Appendix: the kernel schedule, not the tile



One V100. The x–y tile is fixed at 16×8; only the order in which a block walks z changes.

Nsight counters, full-volume → plane-streamed

	n=61	n=97
thread blocks launched	2,048 → 256	8,125 → 637
executed instructions	2.9× fewer	3.4× fewer
global-load sectors	2.6–4.2× fewer	

Each plane is fetched from global memory *once* and reused down the segment.

This does not replace s-step; it composes with it.

Appendix: when does the problem stop fitting on one card?

FP64 footprint	n=61	n=337
unknowns N (millions)	0.23	38.3
one state vector (MB)	1.8	306
Krylov basis, m=39 (GB)	0.07	11.9
share of a 16 GB V100	0.4%	**75%**

At $n = 337$ the solve converges but is still not accepted

Maximum $m = 32$, residual 9.85×10^{-9}: the *convergence* gate passes. The *boundary-state* gate fails, at 6.40×10^{-5}, and the cause is scaling rather than convergence. The fixed augmentation leaves $\|\mathrm{B}\|_1 = 4.15 \times 10^{10}$, so $\widetilde{\mathrm{A}}$ is badly scaled and the three-component forcing tail loses its digits. A power-of-two similarity rescaling, $\eta = 2^{-36}$, brings $\|\mathrm{B}\|_1$ to 0.603 and the tail error to 4.41×10^{-13}, and the run then passes.

The original stop remains the reported result.

Appendix: what a run has to pass

Gate	What it checks	Tolerance
conditioning	$\kappa_2(Z_q)$ before every factorization	$u^{-1/2} \approx 9.5 \times 10^7$
convergence	endpoint defect η_m, at every step	10^{-8}
boundary state	the forcing tail against its closed form	10^{-8}
single-GPU	distributed state against a one-GPU run	10^{-13} $\|\|$state$\|\|$
referee	against an Al-Mohy–Higham solve	stated per arm

A result is reported only if **all five** pass. Each is a differentnquantity, so tightening one does not repair another: the $n = 337$ probe converges and still fails at *boundary state*.